

From Data to Dashboard

An End-to-End Python & Streamlit Workbook

Building interactive bank portfolio dashboards
with pandas, Plotly, and Streamlit

Training Workbook · Sample / Synthetic Data

Contents

1. The big picture: how the pieces fit together
2. Setting up your environment (recap)
3. Loading and inspecting data with pandas
4. Cleaning and exploring the data
5. Calculating banking KPIs
6. Static charts with Matplotlib
7. Interactive charts with Plotly
8. Building the Streamlit dashboard, piece by piece
9. Running the dashboard
10. Connecting to real data sources (SQL Server)
11. Sharing the dashboard with others
12. Troubleshooting reference
13. Glossary of terms

1 The Big Picture

Before writing any code, it helps to see the whole pipeline in one view. Every dashboard you'll ever build follows this same shape:

```
Data Source (CSV, SQL Server, Excel)
|
v
Python reads and cleans the data (pandas)
|
v
Python calculates KPIs and summaries (pandas / numpy)
|
v
Python builds charts (matplotlib / plotly)
|
v
A dashboard framework wraps it all (streamlit)
|
v
A web page in your browser: KPIs, filters, charts, tables
```

The important thing to understand: **Streamlit does not replace pandas or Plotly** — it just wraps your normal Python analysis code in a web layout so a non-technical user can view and filter it through a browser, with no code visible to them.

Mental model: think of a Jupyter notebook as your workshop (where you figure things out, cell by cell) and a Streamlit dashboard as the showroom (a polished version of the same logic that other people can walk through).

2 Setting Up Your Environment (Recap)

This assumes you've already installed Git, VS Code, and Anaconda. Quick recap of what each tool is for:

Tool	Purpose
Anaconda	Manages Python itself and your packages (pandas, streamlit, etc.) in isolated environments
VS Code	Where you write and run your code
Git / GitHub	Version control — tracks changes to your code and lets you back it up / share it

Install the packages you'll need

Open a terminal in VS Code (linked to your Anaconda environment) and run:

```
pip install pandas numpy matplotlib plotly streamlit jupyter notebook openpyxl
```

Confirm it worked:

```
streamlit --version
python -c "import pandas, plotly; print('OK')"
```

You should see a version number and `OK` printed with no errors.

3 Loading and Inspecting Data

Every dashboard starts with getting data into a pandas **DataFrame** — pandas' name for a table with rows and columns, similar to an Excel sheet but scriptable.

Reading a CSV file

```
import pandas as pd

# parse_dates tells pandas to treat that column as real dates,
# not plain text -- this matters later for time-based charts
df = pd.read_csv("data/bank_loan_portfolio.csv", parse_dates=["Disbursement_Date"])

df.head()  # shows the first 5 rows
```

Quick health checks

```
# (rows, columns)
df.shape

# column names, data types, non-null counts
df.info()

# summary statistics for numeric columns
df.describe()
```

Habit worth building: always run `.shape`, `.info()`, and `.head()` immediately after loading any new dataset. It takes 10 seconds and catches most data problems (wrong column count, unexpected data types, missing values) before you go any further.

4 Cleaning and Exploring the Data

Check for missing values

```
df.isnull().sum()
```

Look at categorical columns

```
print("Branches:", df['Branch'].unique())
print("Products:", df['Product'].unique())
```

Filtering rows

```
# All loans in Nairobi CBD
nairobi_loans = df[df['Branch'] == 'Nairobi CBD']

# All loans more than 30 days overdue
overdue = df[df['DPD'] > 30]

# Combine conditions with & (and) or | (or) -- note the brackets around each condition
risky_sme = df[(df['Customer_Segment'] == 'SME') & (df['DPD'] > 30)]
```

Grouping and summarizing

`.groupby()` is the single most useful pandas method for dashboards — it's how you turn thousands of individual loan rows into one summary row per branch, per product, per month, etc.

```
# Total outstanding balance per branch
df.groupby('Branch')['Outstanding_Balance'].sum()

# Multiple aggregations at once
df.groupby('Branch').agg(
    Number_of_Loans=('Customer_ID', 'count'),
    Total_Balance=('Outstanding_Balance', 'sum'),
    Avg_DPD=('DPD', 'mean')
)
```

5 Calculating Banking KPIs

This is where raw data becomes management information — the same numbers that will end up as KPI cards on the dashboard.

Total loan book

```
total_book = df['Outstanding_Balance'].sum()
print(f"Total Outstanding Loan Book: KES {total_book:,.0f}")
```

Default rate

```
# Proportion of loans more than 30 days past due
default_rate = (df['DPD'] > 30).sum() / len(df)
print(f"Default Rate: {default_rate:.1%}")
```

NPL ratio (using IFRS9 staging)

```
npl_balance = df.loc[
    df['IFRS9_Stage'] == 'Stage 3 - Non-Performing', 'Outstanding_Balance'
].sum()

npl_ratio = npl_balance / total_book
print(f"NPL Ratio: {npl_ratio:.1%}")
```

Portfolio breakdown by risk stage

```
stage_summary = df.groupby('IFRS9_Stage').agg(
    Number_of_Loans=('Customer_ID', 'count'),
    Outstanding_Balance=('Outstanding_Balance', 'sum')
).reset_index()

stage_summary
```

Note on real IFRS 9 calculations: this workbook uses a simplified DPD-based staging rule for teaching purposes. Real ECL / IFRS 9 provisioning involves PD, LGD, EAD models and forward-looking macroeconomic overlays — treat this as a dashboard-building exercise, not a provisioning methodology.

6 Static Charts with Matplotlib

Matplotlib is the foundational Python plotting library. Good for reports, slides, and printed output — but the charts are static images, not interactive.

Bar chart

```
import matplotlib.pyplot as plt

product_summary = df.groupby('Product')['Outstanding_Balance'].sum().sort_values(ascending=False)

plt.figure(figsize=(8, 5))
product_summary.plot(kind='bar', color='#1f4e79')
plt.title('Outstanding Balance by Product')
plt.ylabel('Outstanding Balance (KES)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Pie chart

```
segment_summary = df.groupby('Customer_Segment')['Outstanding_Balance'].sum()

plt.figure(figsize=(6, 6))
plt.pie(segment_summary, labels=segment_summary.index, autopct='%1.1f%%')
plt.title('Portfolio Mix by Customer Segment')
plt.show()
```


7 Interactive Charts with Plotly

Plotly charts run in a browser and let the viewer hover, zoom, and pan. This is the same charting library used inside the Streamlit dashboard, so what you build here plugs directly into the app later.

Interactive bar chart

```
import plotly.express as px

fig = px.bar(
    product_summary.reset_index(),
    x='Product',
    y='Outstanding_Balance',
    color='Product',
    title='Outstanding Balance by Product',
    text_auto='.2s'
)
fig.show()
```

Trend line over time

```
trend = (
    df.set_index('Disbursement_Date')
      .resample('ME')['Disbursed_Amount']
      .sum()
      .reset_index()
)

fig = px.line(trend, x='Disbursement_Date', y='Disbursed_Amount', markers=True)
fig.show()
```

Box plot (spread of risk by branch)

```
fig = px.box(df, x='Branch', y='DPD', color='Branch', title='DPD Distribution by Branch')
fig.show()
```

8 Building the Streamlit Dashboard

Streamlit turns a plain Python script into a web app. It reruns your entire script top-to-bottom every time a user interacts with a filter or button — that's the whole mechanism. There's no separate frontend to build.

8.1 Page setup

```
import streamlit as st
import pandas as pd
import plotly.express as px

# Must be the first Streamlit command in the script
st.set_page_config(
    page_title="Bank Loan Portfolio Dashboard",
    page_icon="🏦",
    layout="wide"
)
```

8.2 Loading data with caching

```
# @st.cache_data means the CSV is read from disk once, not on every
# click -- this is what keeps the dashboard responsive
@st.cache_data
def load_data():
    return pd.read_csv("data/bank_loan_portfolio.csv", parse_dates=["Disbursement_Date"])

df = load_data()
```

8.3 Sidebar filters

```
st.sidebar.header("Filters")

branch_filter = st.sidebar.multiselect(
    "Branch",
    options=sorted(df["Branch"].unique()),
    default=sorted(df["Branch"].unique())
)

filtered_df = df[df["Branch"].isin(branch_filter)]
```

8.4 KPI cards

```
total_book = filtered_df["Outstanding_Balance"].sum()
default_rate = (filtered_df["DPD"] > 30).sum() / len(filtered_df)

col1, col2 = st.columns(2)
col1.metric("Total Loan Book", f"KES {total_book:,.0f}")
col2.metric("Default Rate", f"{default_rate:.1%}")
```

8.5 Embedding a chart

```
fig = px.bar(
    filtered_df.groupby("Product")["Outstanding_Balance"].sum().reset_index(),
    x="Product", y="Outstanding_Balance"
)
st.plotly_chart(fig, use_container_width=True)
```

8.6 Data table and download button

```
st.dataframe(filtered_df, use_container_width=True, hide_index=True)

csv = filtered_df.to_csv(index=False).encode("utf-8")
st.download_button("Download CSV", data=csv, file_name="filtered_loans.csv")
```

How each notebook concept maps to Streamlit

In a notebook	In Streamlit

<code>df.head()</code>	<code>st.dataframe(df)</code> — live, scrollable, sortable
<code>print(f"...")</code>	<code>st.metric("Label", value)</code> — styled KPI card
<code>fig.show()</code>	<code>st.plotly_chart(fig)</code> — embedded in the page
Manually re-running cells to filter	<code>st.sidebar.multiselect(...)</code> — the user filters live in-browser

9 Running the Dashboard

Step 1. Open a terminal in your project folder (VS Code terminal or Anaconda Prompt).

Step 2. Install requirements:

```
pip install -r requirements.txt
```

Step 3. Run:

```
streamlit run dashboard.py
```

Step 4. What to expect:

```
Local URL: http://localhost:8501  
Network URL: http://192.168.1.15:8501
```

Your default browser opens automatically to that address. If it doesn't, copy the **Local URL** and paste it into your browser manually.

Step 5. To stop the app, return to the terminal and press `Ctrl + C`.

Every time you change a filter, click a button, or type into a widget, Streamlit re-runs the whole script from top to bottom (using the cached data, so it stays fast) and redraws the page. This is different from a notebook, where you control exactly which cell reruns.

10 Connecting to Real Data Sources

Once you're comfortable with the CSV version, the only thing that changes to use real data is the `load_data()` function — everything downstream (KPIs, charts, filters) keeps working unchanged, since it's built on column names, not the data source.

SQL Server example

```
import pyodbc

@st.cache_data
def load_data():
    conn = pyodbc.connect(
        "DRIVER={SQL Server};SERVER=your_server;DATABASE=your_db;"
        "Trusted_Connection=yes;"
    )
    df = pd.read_sql("""
        SELECT Branch, Product, Outstanding_Balance, DPD,
               Disbursement_Date, Customer_Segment
        FROM Loan_Portfolio
        """, conn)
    return df
```

Excel file example

```
@st.cache_data
def load_data():
    return pd.read_excel("loan_data.xlsx", sheet_name="Portfolio")
```

Important: if your real column names differ from the sample (e.g. `Balance` instead of `Outstanding_Balance`), update every reference to that column name throughout the script — a quick `Ctrl+H` (Find & Replace) in VS Code handles this in seconds.

11 Sharing the Dashboard

`streamlit run` only starts a server on **your own laptop** — nothing is uploaded anywhere automatically. What "sharing" means depends on the option you choose:

Method	Who can see it	Good for
Network URL <code>http://192.168.x.x:8501</code>	Anyone on the same office Wi-Fi/LAN, while your laptop stays on and the app keeps running	Quick demo to a colleague nearby
Streamlit Community Cloud (share.streamlit.io)	Anyone with the public link, anywhere, anytime	Public / demo data only — never real customer data
Internal company server / VM	Anyone on NCBA's internal network / VPN	The correct option once working with real banking data — requires IT to provision and host it

Deploying to Streamlit Community Cloud (demo data only)

1. Push your project folder to a GitHub repository.
2. Go to **share.streamlit.io** and sign in with GitHub.
3. Click **New app**, select your repo and `dashboard.py`.
4. It deploys automatically and gives you a public URL to share.

12 Troubleshooting Reference

Problem	Likely cause / fix
<code>streamlit: command not found</code>	Your environment isn't activated, or packages weren't installed in the environment VS Code is using. Check <code>Python: Select Interpreter</code> .
Browser doesn't open automatically	Manually visit <code>http://localhost:8501</code>
Charts don't update after changing a filter	Normal — Streamlit reruns the script on every interaction, wait a moment
<code>ModuleNotFoundError: No module named 'plotly'</code>	Run <code>pip install -r requirements.txt</code> again in the correct terminal/environment
<code>FileNotFoundError</code> for the CSV	Your terminal's current folder isn't the project root — <code>cd</code> into the folder containing <code>dashboard.py</code>
Dashboard is slow with large data	Make sure <code>@st.cache_data</code> is on your loading function; avoid recomputing summaries inside chart code unnecessarily

Term	Meaning
DataFrame	pandas' table structure — rows and columns, like a spreadsheet
DPD	Days Past Due — how overdue a loan payment is
NPL	Non-Performing Loan — a loan in serious default, typically 90+ DPD
IFRS 9 Stage	Risk classification bucket (Stage 1 performing, Stage 2 underperforming, Stage 3 non-performing) used for loan-loss provisioning
groupby	pandas method that aggregates rows into per-category summaries
KPI	Key Performance Indicator — a headline number summarizing performance
Widget	An interactive element in Streamlit — a filter, button, slider, etc.
Caching (<code>@st.cache_data</code>)	Storing the result of a slow function (like reading a file) so it isn't repeated unnecessarily
localhost	Your own computer, as a network address — only reachable from the same machine
Deployment	Making an app run somewhere persistent (a server) rather than only on your laptop